

# Circuit Documentation

## 1. Circuit Summary

This circuit is an Arduino UNO-based fire-detection, fire-extinguishing, and navigation system. It combines:

- Three flame sensors for detecting fire direction.
- Two HC-SR04 ultrasonic sensors:
  - One for front obstacle and fire-distance detection.
  - One for monitoring water-tank level.
- Two infrared sensors for line following.
- Two DC gear motors controlled by an L298N motor driver.
- Two servos:
  - One operates a water valve.
  - One rotates an ultrasonic sensor for obstacle scanning.
- Red, yellow, and green LEDs that indicate the water level.
- Two 9 V batteries used in the motor-power network.
- An Arduino UNO running the complete control program.

The Arduino monitors the sensors and selects between two primary operating modes:

1. **Fire-extinguishing mode**

When a flame sensor detects fire, the vehicle approaches the fire while maintaining a safe distance. It then stops and opens the water valve if sufficient water is available.

2. **Hybrid maze/navigation mode**

When no fire is detected, the vehicle follows an infrared line. If an obstacle is detected within the programmed distance, the radar servo scans left and right, and the vehicle turns toward the clearer direction.

## Important Electrical Observations

The supplied netlist contains several connections that should be reviewed before constructing the circuit:

- The L298N ENA and ENB pins are connected directly to the Arduino 5 V rail. Motor speed control through PWM is therefore not used.
- The L298N 5V and 5VEN pins are not shown as connected. The required logic-power configuration depends on the specific L298N module. These pins must be wired according to the module datasheet.
- The two 9 V batteries are connected together in a potentially unsafe and electrically inconsistent arrangement:
  - One battery positive terminal connects to the L298N 12V input.
  - Its negative terminal connects to ground.
  - The second battery positive terminal connects to ground.
  - The second battery negative terminal connects to the first battery positive terminal.
- The DC motors are rated for 3–6 V, while the motor-driver supply is connected to a 9 V battery. Applying the full battery voltage to these motors may damage them or cause excessive heating. A suitable regulated motor supply is recommended.

- The two motors are connected to opposite L298N channels with opposite terminal polarity. This may be intentional to make both motors rotate in the same physical direction.
  - The analog outputs A0 of the flame sensors are not connected. The circuit uses only their digital outputs.
  - Both HC-SR04 sensors share the Arduino 5 V and ground rails, but use separate trigger and echo signals.
  - Servo power is supplied from the Arduino 5 V rail in the supplied netlist. Servos can produce current surges; a separate regulated 5 V supply with a common ground is recommended.
- 

## **2. Component List**

### **Arduino UNO**

The Arduino UNO is the central controller. It reads the flame, ultrasonic, and infrared sensors, controls the LEDs and L298N motor driver, and generates servo-control signals.

---

### **Green LED**

The green LED indicates that the measured water level is high or full according to the program thresholds.

---

### **Red LED**

The red LED indicates that the measured water level is low.

---

### **Yellow LED**

The yellow LED indicates an intermediate water level.

---

### **220 $\Omega$ Resistor**

This resistor limits current through the green LED.

---

### **220 $\Omega$ Resistor (1)**

This resistor limits current through the yellow LED.

---

## **220 $\Omega$ Resistor (2)**

This resistor limits current through the red LED.

---

## **Flame Sensor**

This sensor provides a digital flame-detection output to Arduino digital pin D5. Its analog output is not connected.

---

## **Flame Sensor (1)**

This sensor provides a digital flame-detection output to Arduino digital pin D6. Its analog output is not connected.

---

## **Flame Sensor (2)**

This sensor provides a digital flame-detection output to Arduino digital pin D7. Its analog output is not connected.

---

## **DC Gear Motor**

This motor is connected to motor-driver outputs OUT3 and OUT4.

---

## **DC Gear Motor (1)**

This motor is connected to motor-driver outputs OUT1 and OUT2.

---

## **Servo**

This servo operates the water valve and is controlled by Arduino D12.

---

## **Servo (1)**

This servo rotates the navigation ultrasonic sensor and is controlled by Arduino A3, which is used as a digital output.

---

## **9 V Battery**

This battery supplies the L298N motor-driver supply input through its positive terminal and is connected to the system ground through its negative terminal.

---

## **9 V Battery (1)**

This battery is connected into the motor-power network. Its wiring should be reviewed carefully because it is connected between the other battery terminals and ground in the supplied netlist.

---

## **L298N Motor Driver**

The L298N provides dual H-bridge motor control. Its input signals come from Arduino pins D8–D11, and its outputs drive the two DC motors.

---

## **HC-SR04 Ultrasonic Sensor**

This sensor measures the front distance for navigation and fire-approach control. It uses Arduino D13 for trigger and A1 for echo.

---

## **HC-SR04 Ultrasonic Sensor (1)**

This sensor measures the water-tank distance. It uses Arduino A0 for trigger and A2 for echo.

---

## **Infrared Sensor**

This sensor provides the right-side line-following signal to Arduino A5.

---

## **Infrared Sensor (1)**

This sensor provides the left-side line-following signal to Arduino A4.

---

## **Infrared Sensor (2)**

This sensor provides an additional infrared signal to Arduino A2. As wired, this signal shares Arduino A2 with the water-level ultrasonic echo and is therefore electrically conflicted.

---

# **3. Component Wiring Details**

The following sections describe the connections shown in the supplied netlist. Connections not listed in the netlist are treated as unconnected.

## Arduino UNO

### Power

- 5V:
  - HC-SR04 Ultrasonic Sensor
  - HC-SR04 Ultrasonic Sensor (1)
  - Both servos
  - All three flame sensors
  - All three infrared sensors
  - L298N ENA
  - L298N ENB
- GND:
  - L298N GND
  - Negative terminal of the primary 9 V battery
  - All three LED current-limiting resistors
  - Both HC-SR04 sensors
  - Both servos
  - All three flame sensors
  - All three infrared sensors

### Signal Connections

- D2 → red LED anode
- D3 → yellow LED anode
- D4 → green LED anode
- D5 → first flame sensor D0
- D6 → second flame sensor D0
- D7 → third flame sensor D0
- D8 → L298N IN1
- D9 → L298N IN2
- D10 → L298N IN3
- D11 → L298N IN4
- D12 → water-valve servo PWM
- D13 → navigation HC-SR04 TRIG
- A0 → water-level HC-SR04 TRIG
- A1 ← navigation HC-SR04 ECHO
- A2 ← water-level HC-SR04 ECHO
- A2 ← one infrared sensor OUT
- A3 → radar servo PWM
- A4 ← one infrared sensor OUT
- A5 ← one infrared sensor OUT

### Unconnected Arduino Pins

The following Arduino pins are not shown as connected:

- UNUSED
- IOREF
- Reset
- 3.3V
- Vin
- AREF
- SCL
- SDA
- D0
- D1

---

## Green LED

- Anode → Arduino D4
- Cathode → pin 1 of the 220  $\Omega$  green-LED resistor

---

## Red LED

- Anode → Arduino D2
- Cathode → pin 1 of the 220  $\Omega$  red-LED resistor

---

## Yellow LED

- Anode → Arduino D3
- Cathode → pin 1 of the 220  $\Omega$  yellow-LED resistor

---

## 220 $\Omega$ Resistor

This resistor is used with the green LED.

- pin1 → green LED cathode
- pin2 → system ground

---

## 220 $\Omega$ Resistor (1)

This resistor is used with the yellow LED.

- pin1 → yellow LED cathode
  - pin2 → system ground
-

## 220 $\Omega$ Resistor (2)

This resistor is used with the red LED.

- pin1 → red LED cathode
  - pin2 → system ground
- 

## Flame Sensor

- VCC → Arduino 5 V
  - GND → system ground
  - D0 → Arduino D7
  - A0 → unconnected
- 

## Flame Sensor (1)

- VCC → Arduino 5 V
  - GND → system ground
  - D0 → Arduino D6
  - A0 → unconnected
- 

## Flame Sensor (2)

- VCC → Arduino 5 V
  - GND → system ground
  - D0 → Arduino D5
  - A0 → unconnected
- 

## DC Gear Motor

- V+ → L298N OUT3
  - V- → L298N OUT4
- 

## DC Gear Motor (1)

- V+ → L298N OUT2
  - V- → L298N OUT1
- 

## Water-Valve Servo

- VCC → Arduino 5 V

- GND → system ground
  - PWM → Arduino D12
- 

## **Radar Servo**

- VCC → Arduino 5 V
- GND → system ground
- PWM → Arduino A3

Arduino A3 is used as a digital servo-control output by the Servo library.

---

## **Primary 9 V Battery**

- Positive → L298N 12V
  - Negative → system ground
- 

## **9 V Battery (1)**

- Positive → the positive terminal of the primary 9 V battery
- Negative → the positive terminal of the other 9 V battery

This arrangement should not be used without verifying battery polarity and the intended power architecture. Connecting batteries in this manner can cause excessive current, battery damage, or unsafe operating conditions.

---

## **L298N Motor Driver**

### **Power**

- 12V → positive terminal of the primary 9 V battery
- GND → Arduino ground and system ground
- ENA → Arduino 5 V
- ENB → Arduino 5 V

### **Logic Inputs**

- IN1 ← Arduino D8
- IN2 ← Arduino D9
- IN3 ← Arduino D10
- IN4 ← Arduino D11

### **Motor Outputs**

- OUT1 → V- of DC Gear Motor (1)
- OUT2 → V+ of DC Gear Motor (1)



- OUT3 → V+ of DC Gear Motor
- OUT4 → V- of DC Gear Motor

### **Unconnected L298N Pins**

- 5V is not listed as connected.
- 5VEN is not listed as connected.

The appropriate state of 5VEN and the required connection of 5V depend on the specific L298N module. Follow the module's schematic and do not assume that an unconnected logic-supply pin is acceptable.

---

### **Navigation HC-SR04 Ultrasonic Sensor**

This sensor is used for front obstacle detection and fire-distance measurement.

- VCC → Arduino 5 V
- GND → system ground
- TRIG → Arduino D13
- ECHO → Arduino A1

---

### **Water-Level HC-SR04 Ultrasonic Sensor**

This sensor measures the distance to the water surface.

- VCC → Arduino 5 V
- GND → system ground
- TRIG → Arduino A0
- ECHO → Arduino A2

---

### **Infrared Sensor**

- VCC → Arduino 5 V
- GND → system ground
- OUT → Arduino A5

---

### **Infrared Sensor (1)**

- VCC → Arduino 5 V
- GND → system ground
- OUT → Arduino A4

---

### **Infrared Sensor (2)**

- VCC → Arduino 5 V
- GND → system ground
- OUT → Arduino A2

## Wiring Conflict

Arduino A2 is already connected to the water-level HC-SR04 ECHO signal. The infrared sensor output and ultrasonic echo output must not share this pin. One of these signals should be moved to an unused Arduino pin, and the software must be updated accordingly.

---

## 4. Documented Code

### 4.1 Program Purpose

The Arduino program controls a mobile fire-extinguishing robot. It:

- Displays water level using three LEDs.
- Detects fire direction using three flame sensors.
- Measures distance using two ultrasonic sensors.
- Follows an infrared line when no fire is present.
- Avoids obstacles using a scanning servo.
- Controls two DC motors through the L298N.
- Opens and closes a water valve using a servo.
- Prevents valve opening when the water level is too low.

---

### 4.2 Included Library

```
#include <Servo.h>
```

The Servo library generates the control pulses required by both servos.

---

### 4.3 Pin Assignments

#### LED Pins

```
const int LED_RED = 2;  
const int LED_YELLOW = 3;  
const int LED_GREEN = 4;
```

The LEDs are controlled as digital outputs:

- D2: red LED
- D3: yellow LED
- D4: green LED

## Flame Sensor Pins

```
const int FLAME_LEFT = 5;  
const int FLAME_CENTER = 6;  
const int FLAME_RIGHT = 7;
```

The program reads these pins digitally. A flame is considered detected when the sensor output is LOW.

## Motor-Control Pins

```
const int IN1 = 8;  
const int IN2 = 9;  
const int IN3 = 10;  
const int IN4 = 11;
```

These four outputs control the direction of the two L298N H-bridge channels.

## Servo Pins

```
const int VALVE_SERVO_PIN = 12;  
const int RADAR_SERVO_PIN = A3;
```

- D12 controls the water-valve servo.
- A3 controls the radar servo.

## Ultrasonic Sensor Pins

```
const int NAV_TRIG_PIN = 13;  
const int NAV_ECHO_PIN = A1;
```

```
const int WATER_TRIG_PIN = A0;  
const int WATER_ECHO_PIN = A2;
```

The navigation ultrasonic sensor uses D13 and A1. The water-level ultrasonic sensor uses A0 and A2.

### Infrared Sensor Pins

```
const int IR_LEFT_PIN = A4;  
const int IR_RIGHT_PIN = A5;
```

The code expects the left and right line-following sensors on A4 and A5. The supplied wiring also connects another infrared sensor output to A2, which conflicts with the water-level ultrasonic echo input.

---

## 4.4 Operating Constants

```
const int VALVE_CLOSED = 180;  
const int VALVE_OPEN = 90;
```

These servo positions define the closed and open positions of the water valve.

```
const int RADAR_LEFT = 160;  
const int RADAR_CENTER = 90;  
const int RADAR_RIGHT = 20;
```

These positions control the radar servo during obstacle scanning.

```
const int WATER_DIST_FULL = 5;  
const int WATER_DIST_MID = 15;  
const int SAFE_FIRE_DISTANCE = 20;  
const int OBSTACLE_DISTANCE = 10;
```

The distance thresholds are:

- Less than or equal to 5 cm: high/full water indication.

- Between 5 cm and 15 cm: medium water indication.
- Greater than 15 cm: low water indication.
- The robot stops approaching a fire at 20 cm.
- An obstacle is recognized at 10 cm or less.

The water-level interpretation assumes that a smaller measured distance means a higher water level.

---

## 4.5 Setup Routine

The setup() function:

1. Configures LEDs as outputs.
2. Configures flame sensors as inputs.
3. Configures motor-driver control pins as outputs.
4. Configures infrared sensor pins as inputs.
5. Configures ultrasonic trigger and echo pins.
6. Attaches both servos.
7. Sets the water valve to its closed position.
8. Centers the radar servo.
9. Starts serial communication at 9600 baud.

```
waterValve.attach(VALUE_SERVO_PIN);  
waterValve.write(VALUE_CLOSED);
```

```
radarServo.attach(RADAR_SERVO_PIN);  
radarServo.write(RADAR_CENTER);
```

```
Serial.begin(9600);
```

---

## 4.6 Main Loop

The main loop first updates the water-level LEDs:

```
updateWaterLevelLEDs();
```

It then reads the three flame sensors:

```
bool leftFire  = (digitalRead(FLAME_LEFT) == LOW);  
bool centerFire = (digitalRead(FLAME_CENTER) == LOW);  
bool rightFire  = (digitalRead(FLAME_RIGHT) == LOW);
```

```
bool fireDetected = (leftFire || centerFire || rightFire);
```

A LOW signal is interpreted as flame detected.

---

## 4.7 Fire-Extinguishing Mode

When any flame sensor detects fire, the radar servo is centered and the navigation ultrasonic sensor measures the distance to the fire.

```
if (distanceToFire > SAFE_FIRE_DISTANCE) {  
    setWaterValve(false);  
  
    if (centerFire) moveForward();  
    else if (leftFire) turnLeft();  
    else if (rightFire) turnRight();  
}
```

The robot:

- Moves forward if the center sensor detects fire.
- Turns left if only the left sensor detects fire.
- Turns right if only the right sensor detects fire.
- Keeps the water valve closed while approaching.

Once the safe distance is reached:

```
else {  
    stopMotors();  
    setWaterValve(true);  
}
```

The robot stops and requests that the water valve open. The valve function still prevents opening if the water tank is below the minimum level.

---

## 4.8 Navigation and Maze Mode

When no fire is detected, the program closes the water valve and checks the navigation ultrasonic sensor.

```
if (frontDistance <= OBSTACLE_DISTANCE && frontDistance > 0) {  
    stopMotors();
```

```
solveObstacleMaze();  
}
```

If an obstacle is detected within 10 cm, the robot stops and performs a left/right distance scan.  
If the path is clear, the robot reads the infrared line sensors:

```
int leftIR = digitalRead(IR_LEFT_PIN);  
int rightIR = digitalRead(IR_RIGHT_PIN);
```

The programmed behavior assumes:

- HIGH means that a black line is detected.
- LOW means that a black line is not detected.

The movement decisions are:

```
if (leftIR == HIGH && rightIR == LOW) {  
    turnLeft();  
}  
else if (leftIR == LOW && rightIR == HIGH) {  
    turnRight();  
}  
else {  
    moveForward();  
}
```

If both sensors have the same state, the robot moves forward.

---

## 4.9 Obstacle-Scanning Routine

The solveObstacleMaze() function rotates the radar servo and measures distances.

```
radarServo.write(RADAR_LEFT);  
delay(300);  
int leftDist = getDistance(NAV_TRIG_PIN, NAV_ECHO_PIN);  
  
radarServo.write(RADAR_RIGHT);  
delay(300);  
int rightDist = getDistance(NAV_TRIG_PIN, NAV_ECHO_PIN);
```

The servo then returns to the center position:

```
radarServo.write(RADAR_CENTER);  
delay(150);
```

The robot turns toward the side with the larger measured distance:

```
if (leftDist > rightDist) {  
    turnLeft();  
    delay(400);  
} else {  
    turnRight();  
    delay(400);  
}
```

After the turn, the motors are stopped.

---

## 4.10 Ultrasonic Distance Measurement

The `getDistance()` function sends a 10-microsecond trigger pulse and measures the duration of the echo pulse.

```
long duration = pulseIn(echoPin, HIGH, 30000);  
int distance = duration * 0.034 / 2;
```

The calculation uses an approximate speed of sound of 0.034 cm/ $\mu$ s.  
If no echo is received, the function returns 999 cm:

```
if (distance == 0) return 999;
```

This causes an absent or timed-out echo to be interpreted as a very large distance.

---

## 4.11 Water-Level LED Control

The water-level routine measures the tank distance and controls the LEDs.

### Low Water Level



```
if (waterDist > WATER_DIST_MID) {  
    digitalWrite(LED_RED, HIGH);  
    digitalWrite(LED_YELLOW, LOW);  
    digitalWrite(LED_GREEN, LOW);  
}
```

The red LED is turned on when the measured distance is greater than 15 cm.

### **Medium Water Level**

```
else if (waterDist > WATER_DIST_FULL && waterDist <= WATER_DIST_MID) {  
    digitalWrite(LED_RED, LOW);  
    digitalWrite(LED_YELLOW, HIGH);  
    digitalWrite(LED_GREEN, LOW);  
}
```

The yellow LED is turned on for distances greater than 5 cm and less than or equal to 15 cm.

### **High or Full Water Level**

```
else {  
    digitalWrite(LED_RED, LOW);  
    digitalWrite(LED_YELLOW, LOW);  
    digitalWrite(LED_GREEN, HIGH);  
}
```

The green LED is turned on at distances of 5 cm or less.

---

## **4.12 Water-Valve Control**

The setWaterValve() function first measures the water level:

```
int waterDist = getDistance(WATER_TRIG_PIN, WATER_ECHO_PIN);
```

If the water level is too low, the valve is forced closed:

```
if (waterDist > WATER_DIST_MID) {  
    waterValve.write(VALUE_CLOSED);  
    return;  
}
```

Otherwise, the valve is opened or closed according to the function argument:

```
if (open) {  
    waterValve.write(VALUE_OPEN);  
} else {  
    waterValve.write(VALUE_CLOSED);  
}
```

---

## 4.13 Motor-Control Functions

### Move Forward

```
void moveForward() {  
    digitalWrite(IN1, HIGH);  
    digitalWrite(IN2, LOW);  
    digitalWrite(IN3, HIGH);  
    digitalWrite(IN4, LOW);  
}
```

### Turn Left

```
void turnLeft() {  
    digitalWrite(IN1, LOW);  
    digitalWrite(IN2, HIGH);  
    digitalWrite(IN3, HIGH);  
    digitalWrite(IN4, LOW);  
}
```

## Turn Right

```
void turnRight() {  
  digitalWrite(IN1, HIGH);  
  digitalWrite(IN2, LOW);  
  digitalWrite(IN3, LOW);  
  digitalWrite(IN4, HIGH);  
}
```

## Stop

```
void stopMotors() {  
  digitalWrite(IN1, LOW);  
  digitalWrite(IN2, LOW);  
  digitalWrite(IN3, LOW);  
  digitalWrite(IN4, LOW);  
}
```

The motor outputs are controlled only for direction. Because ENA and ENB are tied to 5 V, the code does not provide variable-speed PWM control.

---

## 4.14 Main Loop Timing

The program waits 50 ms at the end of every main loop iteration:

```
delay(50);
```

Additional delays are used during servo scanning and obstacle turns. These delays simplify timing but temporarily prevent the Arduino from responding to other sensors.

---

## 5. Recommended Corrections Before Use

1. Rework the 9 V battery wiring and avoid connecting batteries in opposing or unintended series arrangements.

2. Use a regulated 3–6 V supply for the DC gear motors.
3. Confirm the L298N 5V and 5VEN wiring for the exact motor-driver module.
4. Move the additional infrared sensor currently connected to Arduino A2.
5. Update the software pin assignment after moving that infrared sensor.
6. Use a separate regulated 5 V supply for the servos if they draw significant current.
7. Add bulk decoupling capacitors near the motor driver and servo supply.
8. Verify flame-sensor output polarity; the code assumes active-low operation.
9. Confirm the LED polarity and valve-servo angles experimentally before operating the water system.
10. Ensure all external supplies share a suitable common ground with the Arduino logic ground.